Requirements Gathering – Student Performance Dashboard

1. Stakeholder Analysis

The Student Performance Dashboard operates with multiple main stakeholders who possess distinct requirements and expectations for the project.

1.1 Students

The primary data contributors for the system are students who use their academic progress tracking to obtain indirect benefits. Students need to access their academic progress records which contain their scores and attendance data and their performance development throughout the academic year. Students need to discover their academic strengths and weaknesses which will help them achieve better academic results.

1.2 Teachers / Instructors

The system enables teachers to monitor student academic performance across various subjects. Teachers need assessment instruments which help them detect students who need extra support while they observe student development and attendance patterns and review their teaching success.

1.3 School Administrators

Administrators need to access academic performance information which spans all school classes and departments at various times during the day. The system enables organizations to make decisions while it manages policy updates and resource allocation requirements.

1.4 System Developers / Data Analysts

The team handles all system development work which includes system maintenance and system enhancement tasks. The team needs structured data which has been cleaned through data processing along with an operational database design and dependable analytical results.

1.5 Academic Advisors

Advisors use the system to help students by studying performance patterns and suggesting appropriate interventions and support programs.

2. User Stories & Use Cases

2.1 Student Use Cases

The student requires access to his subject scores because it serves as the method to track his academic progress. The student needs to access his attendance records because it shows him how much he has participated. The student needs to see his performance data over different time periods so he can find areas where he needs to improve.

2.2 Teacher Use Cases

The teacher requires access to the highest achieving students in his subject because it helps him identify outstanding students. The teacher needs to review the average student scores for each topic on a monthly basis because this data helps him assess his teaching methods. The teacher needs to review attendance records because it helps him discover when students show signs of disengagement.

2.3 Administrator Use Cases

The administrator requires access to complete school performance data because it helps him make better choices. The administrator needs to evaluate subject performance data because it enables him to find departments that need improvement. The administrator needs to produce academic evaluation meeting reports.

2.4 Analyst Use Cases

The data analyst needs to examine student performance data because it allows him to produce valuable insights. The data analyst needs to transform raw data into a clean format which enables him to perform analysis and create visualizations.

3. Functional Requirements

The system must provide the following functionalities:

- Load student performance data from CSV files using Python.
- Clean and preprocess data (handle missing values, remove duplicates, format dates).
- Categorize student performance (High, Medium, Low).
- Store structured data in a relational database (SQLite or PostgreSQL).
- Design normalized tables for Students, Subjects, Scores, and Attendance.
- Support SQL queries for:
    - Top performers by subject
    - Average scores by month
    - Attendance trends analysis
- Generate data visualizations using Python libraries (Matplotlib / Seaborn).
- Display score trends over time.
- Display attendance distribution and patterns.
- Provide optional interactive dashboard using Streamlit or Plotly Dash.
- Export reports and query results for documentation.

4. Non-Functional Requirements

4.1 Performance Requirements

- The system should efficiently handle large datasets (at least thousands of student records).
- Data processing and query execution should be completed within acceptable time limits (few seconds for standard queries).

4.2 Security Requirements

- Ensure student data is protected and not accessible without proper authorization.
- Prevent unauthorized modification of database records.
- Use secure database connections where applicable.

4.3 Usability Requirements

- The system should have a simple and intuitive interface for dashboards and visualizations.
- Outputs (charts, tables) should be easy to understand for non-technical users such as teachers and administrators.

4.4 Reliability Requirements

- The system should correctly handle missing or inconsistent data without crashing.
- Database operations should maintain data integrity.
- The system should produce consistent results across repeated executions.

4.5 Maintainability Requirements

- Code should be modular and well-structured (separate preprocessing, database, and visualization logic).
- Database schema should be extensible for future enhancements.
- The system should allow easy updates to data sources or analytical features.

4.6 Scalability Requirements

- The architecture should allow future expansion.
- The system should support migration from SQLite to PostgreSQL if needed.